

Задание 7. Проектирование схем коллекций для шардирования данных

Операции:

1. Быстрое создание заказов с одновременным списанием остатков.

2. Поиск истории заказов конкретного пользователя.

3. Отображение статуса заказа.

orders
id
user_id
created_at
products
status
amount
zone

Операции:

1. Частые обновления остатков при покупках.

2. Поиск товаров по категориям и фильтрация по диапазону цен.

3. Описание товара на странице продукта.

products
id
name
category
price
zone_count
additional

Операции:

1. Создание корзины, когда заходит гость или новый пользователь.

2. Получение текущей корзины по фильтру { session_id, status:"active" } или { user_id, status:"active" }.

3. Добавление или замена товара в корзине.

4. Удаление товара из корзины.

5. Слияние гостевой корзины в пользовательскую, если пользователь залогинится:

1. прочитать гостевую { session_id, status:"active" };

2. добавить её items в корзину { user_id, status:"active" };

3. отметить гостевую как abandoned.

6. Отметка корзины как заказанной.

carts
id
user_id
session_id
items{product_id, quantity}
status
created_at
updated_at
expires_at

К коллекциям orders и products можно применить геошардинг (по ключам zone и zone_count соответственно). Обоснование:

1. Каждый заказ относится к конкретной геоzone. С помощью ключа zone можно разбить коллекцию на шарды по геоzoneам.
2. Кол-во продуктов в каждой геоzone разное. Логично, с помощью ключа zone_count разбить коллeкцию на шарды.

Так же к коллекции product можно применить диапазонное шардирование (по ключу price).

К коллекции carts можно применить хешированное шардирование (по ключу session_id). Это позволит равномерно распределить коллекцию carts по серверам.

Задание 8. Выявление и устранение «горячих» шардов

Набор метрик для отслеживания состояния шардов:

- CPU
- RAM

Необходимо использовать балансировщик нагрузки. Выбрать алгоритм балансировки least connection.

Для каждой метрики установить максимальный порог, при превышении которого должен запускаться процесс создания реплики в автоматическом режиме.

Задание 9. Настройка чтения с реплик и консистентность

Orders		
Операция чтения	Реплика	Обоснование
Поиск истории заказов конкретного пользователя.	secondary	Задержка репликации ≤ 1 мин. Информация не критичная, может обновиться с задержкой.
Отображение статуса заказа.	primary	Статус должен отображаться всегда в актуальном состоянии

Products		
Операция чтения	Реплика	Обоснование
Частые обновления остатков при покупках.	primary	Критично держать в актуальном состоянии кол-во остатков, иначе есть вероятность продать больше, чем осталось товаров
Поиск товаров по категориям и фильтрация по диапазону цен.	secondary	Задержка репликации ≤ 1 мин. Информация не критичная, может обновиться с задержкой.
Описание товара на странице продукта.	secondary	Задержка репликации ≤ 1 мин. Информация не критичная, может обновиться с задержкой.

Carts		
Операция чтения	Реплика	Обоснование
Получение текущей корзины по фильтру { session_id, status:"active" } или { user_id, status:"active" }.	primary	Пользователь должен видеть актуальный список в корзине. Иначе, при добавлении нового товара, он может появиться в корзине с задержкой, что может сказаться на негативном

		опыте пользователя с приложением.
--	--	-----------------------------------

Задание 10. Миграция на Cassandra: модель данных, стратегии репликации и шардирования

Корзина, товары и заказы являются критически важными сущностями с точки зрения целостности и скорости обработки.

Для товаров должны корректно рассчитываться остатки на складах (геораспределение), показываться актуальная цена.

В корзине должны отображаться все добавленные товары для корректного оформления заказа.

Заказ должен находиться в актуальном статусе, чтобы не было проблем с оплатой заказа.

Для товаров необходимо выбрать стратегию восстановления согласованности – Read Repair, т.к. критически важно правильно рассчитывать остатки товаров и их стоимость.

Для корзины и заказов может подойти Hinted Handoff.